SOFTWARE ENGINEERING

Lab 11: Automated Unit Testing and Coverage Analysis

1. Objective

In this laboratory, you act as a Software Quality Assurance (QA) and Test Engineer. You are provided with a complete, backend-only **Ride Management System** that simulates the core workflows of platforms such as Uber and Ola. Your task is to:

- Analyse and study the modular architecture and business logic of each module.
- Develop automated test suites using `pytest` without modifying any production source code.
- Achieve at least 90% line coverage and at least 90% branch coverage using `pytest-cov` / `coverage`.

Source of Code

The `ride_management` code used in this lab is the code attached in the Google Classroom assignment. You must use it as it is, without any modification.

2. Environment Setup

From the `ride_management` root directory, install the test dependencies and verify the tools:

```
pip install -r requirements.txt

pytest --version
coverage --version
```

3. Codebase Architecture and Study Guide

All source code is located under `src/ride_management/`. Study the modules in the following sequence before writing any tests.

Step	Module	File	Core concepts to inspect
1	Exceptions	exceptions.py	Custom error hierarchy derived from *RideManagementError*. Every business error raises a specific exception.
2	Models and Enums	models.py	Dataclasses (Booking, UserProfile, Vehicle, FareBreakdown, Transaction) and enums (BookingStatus, VehicleCategory, DriverStatus, PaymentMethod).
3	Utilities	utils.py	Clock (injectable for time control), IdGenerator (deterministic sequences), validation patterns for email, phone and registration numbers.
4	Location	location.py	Great-circle distance (Haversine formula) and operational boundary box checks.

Step	Module	File	Core concepts to inspect
5	User Management	user.py	Registration, role validation, duplicate checks, wallet credit and debit, activation and deactivation.
6	Vehicle Management	vehicle.py	Vehicle registration, category capacity, age restriction (maximum 12 years), driver association.
7	Driver Management	driver.py	Verification status, rating averages, auto-suspension on low ratings (below 3.0 after 5 trips), availability states.
8	Pricing Engine	pricing.py	Decimal arithmetic, base fare, waiting charges, night surcharge (23:00 to 05:00), peak hours, surge multiplier, coupon expiry and limits, 5% GST.
9	Booking Lifecycle	booking.py	State machine: REQUESTED, DRIVER_ASSIGNED, ACCEPTED, ARRIVED, IN_PROGRESS, COMPLETED, CANCELLED. Illegal transitions and duplicate active rides.
10	Dispatch and Allocation	dispatch.py	Nearest driver within radius, category compatibility, deterministic sorting (distance, rating, ID), driver rejection and retry.
11	Payment Management	payment.py	PaymentGateway interface, MockPaymentGateway, wallet debit, cash collection, UPI and card charges, failure triggers.
12	Cancellation and Refund	cancellation.py	Tiered cancellation fees (grace period up to 2 minutes, after arrival, in progress), driver release, automated refunds.
13	Rating and Feedback	rating.py	Two-way ratings (1 to 5 stars), duplicate rating prevention, average calculation.
14	Notifications	notification.py	Mock SMS and email channels, message log auditing.
15	Analytics	analytics.py	Revenue calculation, driver net earnings (after 20% commission), daily statistics aggregation.
16	System Facade	system.py	RideManagementSystem unifying all services for end-to-end integration tests.

4. Writing Unit Tests

4.1 Required test layout

Create a `tests/` directory at the project root with the following structure:

```
ride_management/
  src/
    ride_management/
  tests/
    conftest.py
    test_user.py
    test_vehicle.py
    test_driver.py
    test_location.py
    test_pricing.py
    test_booking.py
    test_dispatch.py
    test_payment.py
    test_cancellation.py
    test_rating.py
    test_analytics.py
    test_system.py
  requirements.txt
  README.md
```

4.2 Testing strategies

- **Happy paths and error branches.** Test successful flows as well as invalid state transitions (for example, completing an already cancelled booking). Test validation errors such as an invalid email, an invalid phone number, a negative balance and out-of-bounds coordinates.
- **Deterministic time injection.** Use `Clock` to test the night surcharge (23:30 or 02:00) against daytime pricing. Set the time to 09:00 on a weekday to test surge floor pricing. Use `clock.advance_minutes(...)` to test cancellation fee tiers beyond the 2-minute grace period.
- **Mock interfaces.** Use `MockPaymentGateway` with `set_failure_trigger(...)` to test payment failures and `FAIL_REFUND` triggers. Use `MockSmsService(should_succeed=False)` to exercise notification failure branches.
- **Boundary values.** Ratings at 1 and 5, plus illegal values 0 and 6. Vehicle age at the 12-year boundary. Coupon usage limit and expiry date boundaries.
- **Fixtures.** Place shared fixtures (clock, users, vehicles, drivers, gateways) in `conftest.py`.

5. Running Tests and Generating Coverage Reports

Run the following commands from the `ride_management` directory.

5.1 Standard test run

```
pytest
```

5.2 Line and branch coverage in the terminal

```
pytest --cov=src/ride_management --cov-branch --cov-report=term-missing
```

5.3 Interactive HTML coverage report

```
pytest --cov=src/ride_management --cov-branch --cov-report=html
```

Open `htmlcov/index.html` in a browser to inspect the lines and branches that were missed. Missed lines are highlighted in red and partially covered branches in yellow.

6. Submission Requirements

Submit a single PDF report. The report must contain:

– a test case table listing, for each test module, the test ID, the module and behaviour under test, the input or setup, the expected result and the actual result (pass or fail);

– screenshots of the terminal output from the coverage command in Section 5.2, showing the line and branch coverage percentages and the missed-lines column;

– a screenshot of the terminal output from the standard `pytest` run in Section 5.1, showing the pass count.

Name the report using your student ID, for example `202301234_Lab11_Report.pdf`.